UNIVERSITE ABDERRAHMANE MIRA – BEJAIA –

Faculté des Sciences Exactes
Département informatique
Module DevOps

TP N° 2 : Linux & Scripting Bash

Objectifs pédagogiques

- Se repérer dans un système de fichiers Linux et manipuler les permissions ;
- Écrire des scripts Bash (variables, conditions, boucles, fonctions) ;
- Automatiser une tâche récurrente et la planifier avec cron ;
- Comprendre les codes de retour et la gestion d'erreurs ;
- **Versionner son travail avec Git et le publier sur GitHub.**



Exercice 2.0 — Mise en place du dépôt GitHub

1. Créez un dépôt vide tp-bash sur GitHub (sans README).
2. Initialisez un dépôt local, liez-le au dépôt distant, ajoutez un .gitignore.
3. Faites **au moins un commit par exercice**, avec un message explicite.

Exercice 2.1 — Navigation, recherche et permissions

Consigne :

1. Créez une arborescence `tp-bash/logs` et `tp-bash/data` avec quelques fichiers.
2. Listez tous les fichiers `.log` du dossier `logs/`.
3. Recherchez, dans toute l'arborescence, les fichiers modifiés il y a plus de 7 jours.
4. Recherchez le mot « erreur » dans tous les fichiers portant l'extension .log. Ensuite, ajoutez le mot « erreur » au fichier **tp-bash/logs/app1.log**, puis effectuez à nouveau la recherche pour vérifier que le mot est bien détecté.
5. Rendez un script exécutable uniquement par son propriétaire.

Commandes à utiliser : `mkdir -p, ls -lh, find -name -mtime, grep -ri, chmod, chown`

Question de réflexion : Que donne `chmod 644` sur un fichier ? Est-il exécutable par son propriétaire ?

Exercice 2.2 — Premier script Bash : vérification d'espace disque

Consigne : Écrivez un script `check_disk.sh` qui affiche une alerte si l'espace disque utilisé dépasse un seuil donné en argument (80% par défaut).

Commandes/éléments de syntaxe à utiliser :

```
#!/bin/bash
$1, ${1:-valeur_par_défaut}
$(commande)          # substitution de commande
df /
awk '{print $N}'
tr -d '%'
if [ "$VAR" -ge "$SEUIL" ]; then ... fi
echo
exit 0 / exit 1
```

À tester:

```
chmod +x check_disk.sh
./check_disk.sh
./check_disk.sh 10
```

Question de réflexion : Pourquoi utilise-t-on des guillemets autour de `"$UTILISATION"` dans le test `if` ?

Exercice 2.3 — Script de sauvegarde avec boucle et gestion d'erreurs

Consigne : Écrivez `backup.sh` qui archive un dossier, horodate l'archive, et supprime les sauvegardes de plus de 7 jours.

Commandes/éléments de syntaxe à utiliser :

```
set -Eeuo pipefail
date +%Y-%m-%d_%H-%M-%S
mkdir -p
tar -czf archive.tar.gz <dossier>
[ ! -d "$DOSSIER" ]     # test d'existence
>&2                       # redirection vers stderr
for fichier in <motif>; do ... done
find <fichier> -mtime +7
rm
```

Question de réflexion : Que se passe-t-il si vous enlevez `set -Eeuo pipefail` et que `tar` échoue ? Le script continue-t-il à tort ?

Exercice 2.4 — Planification avec cron

Consigne : Planifiez l'exécution de `backup.sh` chaque nuit à 2h00, avec les logs redirigés dans un fichier.

Commandes à utiliser :

```
crontab -e
crontab -l
>> fichier.log 2>&1
tail -f fichier.log
```

Rappel syntaxe cron : `minute heure jour_du_mois mois jour_de_semaine commande`

Question de réflexion : Pourquoi cron nécessite-t-il des chemins absolus alors qu'un script fonctionne très bien depuis votre terminal avec des chemins relatifs ?

Exercice 2.5 — Mini-projet d'intégration (préfiguration CI/CD)

Consigne : Écrivez `ci.sh` qui simule un pipeline minimal : clone/mise à jour d'un dépôt, exécution de "tests", journalisation horodatée, code de sortie explicite.

Commandes/éléments de syntaxe à utiliser :

```
set -Eeuo pipefail
$1                          # argument du script
[ -z "$VAR" ]               # test de variable vide
git clone <url> / git pull
date '+%Y-%m-%d %H:%M:%S'
tee -a <fichier>
fonction() { ... }          # définition de fonction Bash
exit 0 / exit 1
```

Question de réflexion : Quelles étapes ajouteriez-vous pour transformer ce script en un vrai pipeline CI ? Quels seraient les points de défaillance à gérer avec des codes de retour ?

Livrable

Un dossier contenant : `check_disk.sh`, `backup.sh`, `ci.sh` (exécutables), un fichier `backup.log` généré par une exécution réelle, une tâche cron vérifiable (`crontab -l`), un fichier `REPONSES.md` avec les réponses aux questions de réflexion.